KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

Air Pollution Monitoring System using MQ2 Gas Sensor

EXERCISE NO – 04

HARRISH T M – 24MER016

KAPIL P – 24MER022

RAMPRASATH D – 24MER047

SARAVANAKARTHIK S R – 24MER055

SETHUPATHI M – 24MER056

4. Air Pollution Monitoring System

PROJECT OVERVIEW:

The Air Pollution Monitoring System using MQ-2 Gas Sensor is an IoT-based monitoring system designed to detect the presence of smoke and combustible gases in the surrounding environment.

The system uses an ESP8266 Node MCU as the main controller. The MQ-2 gas sensor detects gases such as smoke, LPG, methane, propane and other combustible gases. The sensor output is read by the ESP8266 and processed to determine the air condition.

An OLED/LCD display is used to show the gas sensor value and the current status of the air. Two LED indicators provide visual warnings, while a buzzer provides an audible alert when the detected gas level becomes high.

Objectives:

- To detect harmful gases and smoke.
- To monitor gas levels continuously.
- To display the sensor reading.
- To provide visual indication using LEDs.
- To provide an audible warning using a buzzer.
- To develop a low-cost monitoring system.
- To provide a base for future IoT-based air monitoring.

PROBLEM STATEMENT:

Air pollution and gas leakage can create serious safety problems in industrial areas, workshops, laboratories, kitchens and other environments.

Conventional air-quality monitoring systems can be expensive and may require complicated equipment. A simple and low-cost system is therefore required to continuously monitor the presence of smoke and combustible gases.

The proposed system uses an **MQ-2 gas sensor connected to an ESP8266 Node MCU**. The sensor detects changes in gas concentration and sends the corresponding signal to the controller. The ESP8266 processes the signal and displays the condition on an OLED/LCD.

When the gas level is within the normal range, the system indicates a safe condition. When the level becomes high, the system activates the warning LED and buzzer.

PROPOSED SOLUTION:

The proposed system uses an **MQ-2 Gas Sensor and ESP8266 Node MCU** to continuously monitor the surrounding air.

The MQ-2 sensor detects changes in smoke and combustible gas concentration. It's analog output is connected to the **ESP8266** analog input.

The ESP8266 reads the sensor value and compares it with predefined threshold values.

Working conditions:

Gas level	LED	Buzzer	Display
Low	Green on	Off	Air safe
Medium/High	Red on	On	Warning/Danger

SYSTEM ARCHITECTURE:

The Air Pollution Monitoring System consists of the MQ-2 gas sensor, ESP8266 Node MCU, OLED/LCD display, LED indicators, and buzzer. The MQ-2 sensor detects smoke and combustible gases from the surrounding air. The sensor output is given to the ESP8266 Node MCU, which processes the signal and determines the air-quality status.

The detected gas value is displayed on the OLED/LCD. When the gas level is within the normal range, the green LED indicates a safe condition. When the gas level exceeds the predefined threshold, the red LED and buzzer are activated to alert the user.

Architecture Description

1. **MQ-2 Gas Sensor:** Detects smoke and combustible gases in the surrounding environment.

2. **ESP8266 Node MCU:** Acts as the main controller. It reads and processes the MQ-2 sensor signal.

3. **OLED/LCD Display:** Displays the gas sensor value and air-quality status.

4. **Green LED:** Indicates that the detected gas level is within the selected safe range.

5. **Red LED:** Indicates that the detected gas level has exceeded the selected threshold.

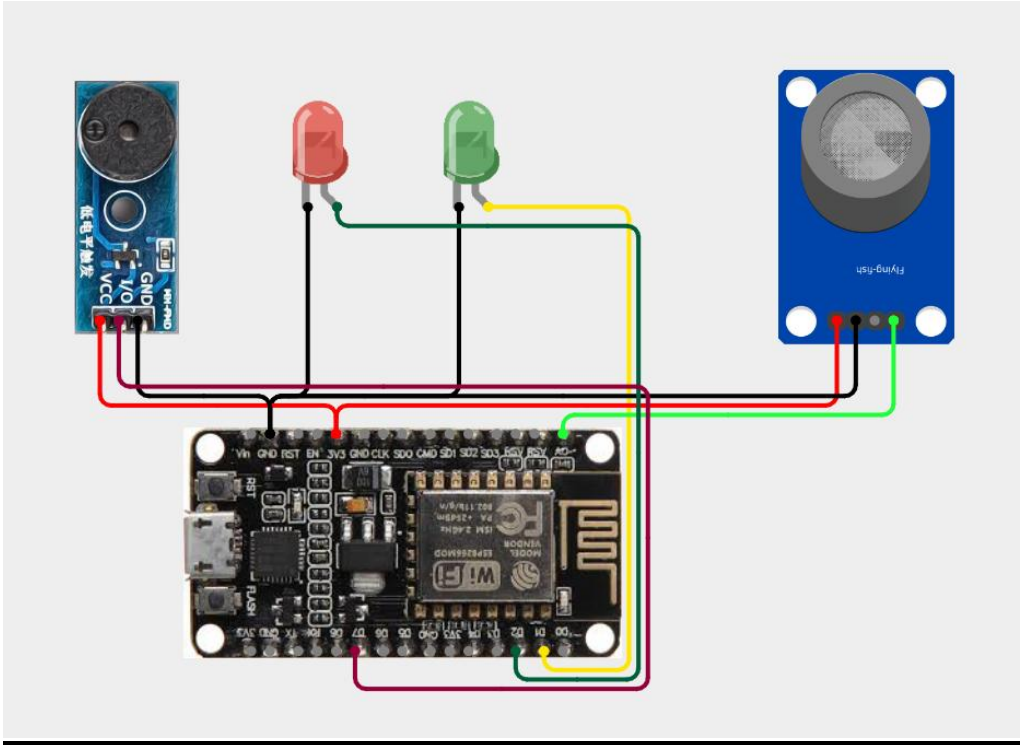
6. **Buzzer:** Produces an audible warning during a high gas-level condition.

7. **Wi-Fi:** The ESP8266 can later transmit the sensor data to an IoT/cloud platform for remote monitoring.

CIRCUIT TABLE:

Component	ESP8266 Node MCU Pin	Function
MQQ-2AO	AO	Analog gas sensor input
OLED SDA	D2/CPIO4	I²C data
OLED SCL	D1/GPIO5	I²C clock
Green LED	D6/GPIO12	Safe indication
Red LED	D7/GPIO13	Danger indication
Buzzer	D5/GPIO14	Alarm
MQ-2 VCC	5V/VIN	Sensor power
MQ-2-GND	GND	Ground
OLED VCC	3.3V	Display power
OLED GND	GND	Ground

CIRCUIT DESIGN:



ALGORITHM:

Algorithm for Air Pollution Monitoring System

- Start the system.
- Initialize the ESP8266 Node MCU.
- Initialize the MQ-2 sensor.
- Initialize the OLED/LCD display.
- Configure the LED and buzzer pins.
- Allow the MQ-2 sensor to warm up.
- Read the analog value from the MQ-2 sensor.
- Display the sensor value on the OLED/LCD.
- Compare the sensor value with the predefined threshold.
- If the value is below the threshold:
    - Turn ON the green LED.
    - Turn OFF the red LED.
    - Turn OFF the buzzer.
    - Display AIR SAFE.
- If the value exceeds the threshold:
    - Turn OFF the green LED.
    - Turn ON the red LED.
    - Turn ON the buzzer.
    - Display AIR POLLUTION / DANGER.
- Repeat the process continuously.
- Stop.

CODING :

```
#include "AdafruitIO_WiFi.h"

#define WIFI_SSID       "Harrish"
#define WIFI_PASSWORD   "Password"



#define IO_USERNAME     "Kapilparamasivam"
#define IO_KEY          "aio_LUjQ50sb43WYuxYc1oLL7ESLA03X"



AdafruitIO_WiFi io(IO_USERNAME, IO_KEY,
                   WIFI_SSID, WIFI_PASSWORD);



AdafruitIO_Feed *gasLevelFeed = io.feed("gas-level");
AdafruitIO_Feed *airStatusFeed = io.feed("air-status");
AdafruitIO_Feed *gasAlertFeed = io.feed("gas-alert");



#define MQ2_PIN       A0

#define GREEN_LED     5       // GPIO5
#define RED_LED       4       // GPIO4
#define BUZZER        13      // GPIO13



#define BUZZER_ON     LOW
#define BUZZER_OFF    HIGH



const int NORMAL_LIMIT = 200;
const int WARNING_LIMIT = 250;



unsigned long lastSendTime = 0;

const unsigned long SEND_INTERVAL = 5000;
```

```
void setup() {

  Serial.begin(115200);

  delay(2000);



  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);


  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  // Buzzer OFF
  digitalWrite(BUZZER, BUZZER_OFF);



  Serial.println();
  Serial.println("");
  Serial.println("    AIR POLLUTION MONITORING SYSTEM");
  Serial.println("    ESP8266 + MQ-2");
  Serial.println("");

  Serial.println("Connecting to Adafruit IO...");

  io.connect();

  while (io.status() < AIO_CONNECTED) {

    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println("Adafruit IO Connected!");
  Serial.println("--------------------------------------");
}



void loop() {

  // Keep Adafruit IO connection alive
  io.run();
```

```
int gasValue = analogRead(MQ2_PIN);

String airStatus;
String gasAlert;



if (gasValue < NORMAL_LIMIT) {

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);

  // Buzzer OFF
  digitalWrite(BUZZER, BUZZER_OFF);

  airStatus = "NORMAL";
  gasAlert = "SAFE";

}




else if (gasValue < WARNING_LIMIT) {

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);

  // Buzzer OFF
  digitalWrite(BUZZER, BUZZER_OFF);

  airStatus = "WARNING";
  gasAlert = "GAS LEVEL HIGH";

}



else {

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);

  airStatus = "DANGER";
  gasAlert = "HIGH GAS / SMOKE DETECTED";

  // Buzzer ON
```

```
    digitalWrite(BUZZER, BUZZER_ON);

    delay(300);

    // Buzzer OFF
    digitalWrite(BUZZER, BUZZER_OFF);

    delay(300);
  }



  Serial.print("MQ-2 Value     : ");
  Serial.println(gasValue);

  Serial.print("Air Status     : ");
  Serial.println(airStatus);

  Serial.print("Alert          : ");
  Serial.println(gasAlert);

  Serial.println("---------------------------------------");



  if (millis() - lastSendTime >= SEND_INTERVAL) {

    lastSendTime = millis();

    Serial.println("Sending data to Adafruit IO...");

    // Send raw MQ-2 value
    gasLevelFeed->save(gasValue);

    // Send air quality status
    airStatusFeed->save(airStatus);

    // Send alert message
    gasAlertFeed->save(gasAlert);

    Serial.println("Data sent successfully!");

    Serial.println("---------------------------------------");
  }


  delay(1000);
}
```

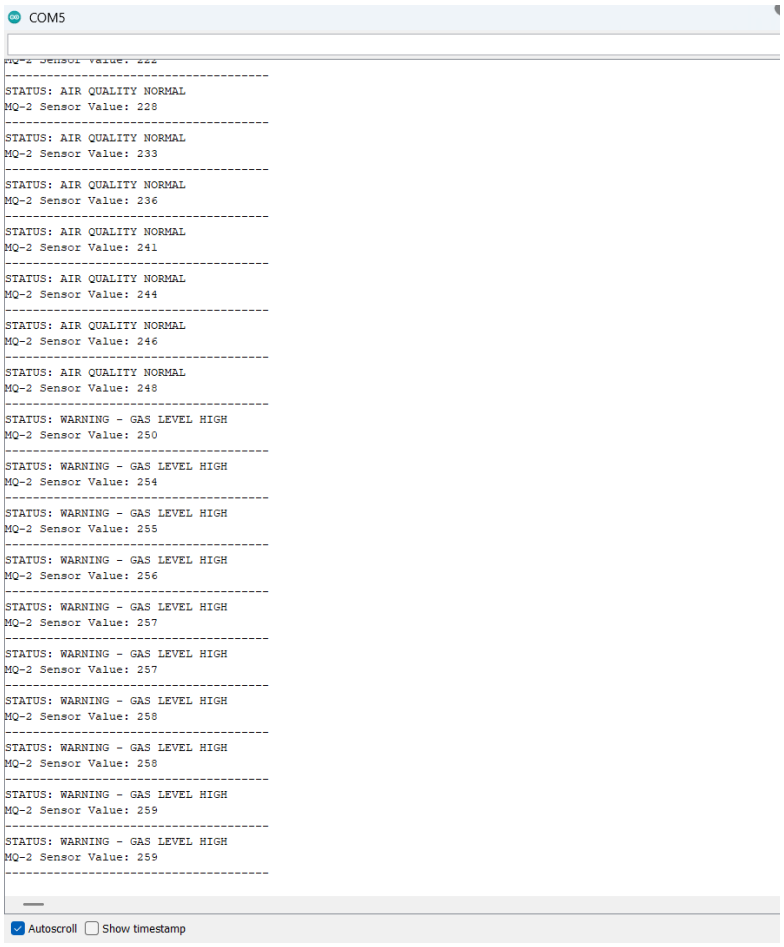
SYSTEM WORKING:

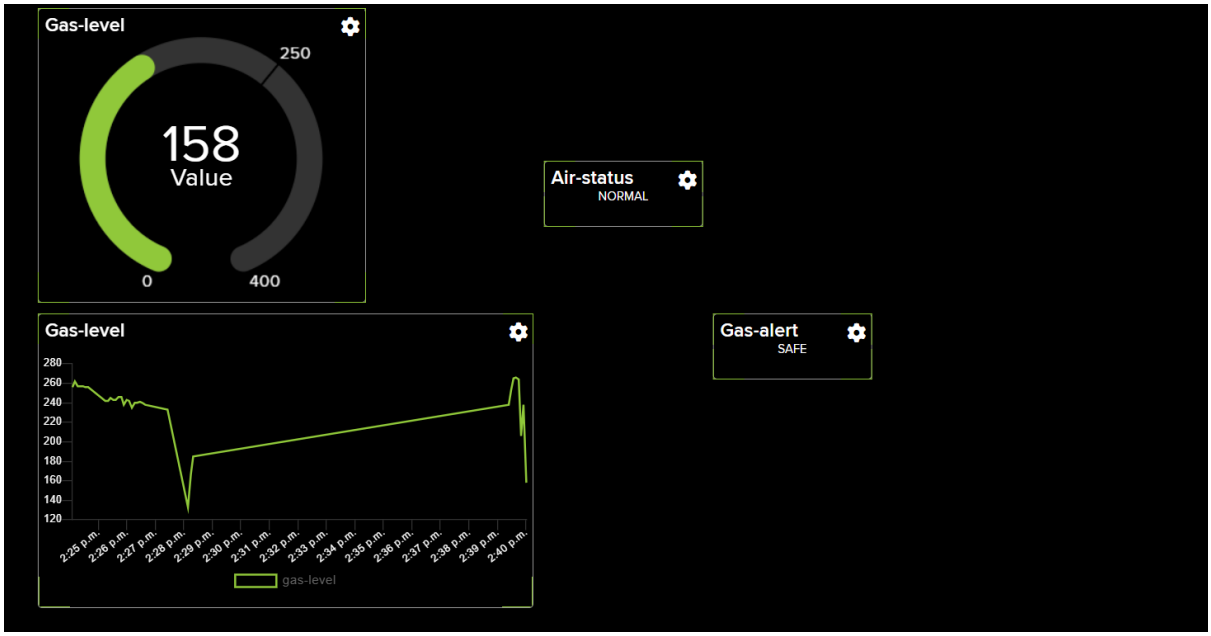
The system starts when power is supplied to the ESP8266 NodeMCU.

The MQ-2 sensor continuously senses smoke and combustible gases present in the surrounding environment.

The sensor generates an analog signal corresponding to its sensing response. The ESP8266 reads this signal through the A0 pin.

The controller then compares the sensor value with the predefined threshold.


COM5
MQ-2 Sensor Value: 222
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 228
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 233
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 236
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 241
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 244
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 246
STATUS: AIR QUALITY NORMAL
MQ-2 Sensor Value: 248
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 250
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 254
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 255
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 256
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 257
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 257
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 258
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 258
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 259
STATUS: WARNING - GAS LEVEL HIGH
MQ-2 Sensor Value: 259
Autoscroll
Show timestamp


Gas-level
250
158
Value
0
400
Air-status
NORMAL
Gas-level
280
260
240
220
200
180
160
140
120
2:25 p.m.
2:26 p.m.
2:27 p.m.
2:28 p.m.
2:29 p.m.
2:30 p.m.
2:31 p.m.
2:32 p.m.
2:33 p.m.
2:34 p.m.
2:35 p.m.
2:36 p.m.
2:37 p.m.
2:38 p.m.
2:39 p.m.
2:40 p.m.
gas-level
Gas-alert
SAFE


Gas-level
250
266
Value
0
400
Air-status
DANGER
Gas-level
280
260
240
220
200
180
160
140
120
2:25 p.m.
2:26 p.m.
2:27 p.m.
2:28 p.m.
2:29 p.m.
2:30 p.m.
2:31 p.m.
2:32 p.m.
2:33 p.m.
2:34 p.m.
2:35 p.m.
2:36 p.m.
2:37 p.m.
2:38 p.m.
2:39 p.m.
gas-level
Gas-alert
HIGH GAS / SMOKE DE

BILL OF MATERIALS:

Component	Quantity
ESP8266 NodeMCU	1
MQ2 Gas Sensor	1
OLED/LCD Display	1
Buzzer	1
LED Indicators	2
Breadboard	1
Jumper Wires	1 Set

—

PROTOTYPE IMPLEMENTATION:

Step 1: Place the ESP8266 NodeMCU on the breadboard.

Step 2: Connect the MQ-2 sensor to the required power supply and ground.

Step 3: Connect the MQ-2 analog output to the ESP8266 A0 input, ensuring the voltage is within the board's ADC input limit.

Step 4: Connect the OLED display using the I²C pins.

Step 5: Connect the green LED to D6.

Step 6: Connect the red LED to D7.

Step 7: Connect the buzzer to D5.

Step 8: Connect the ESP8266 to the computer using the USB cable.

Step 9: Open the Arduino IDE.

Step 10: Select the appropriate ESP8266 NodeMCU board.

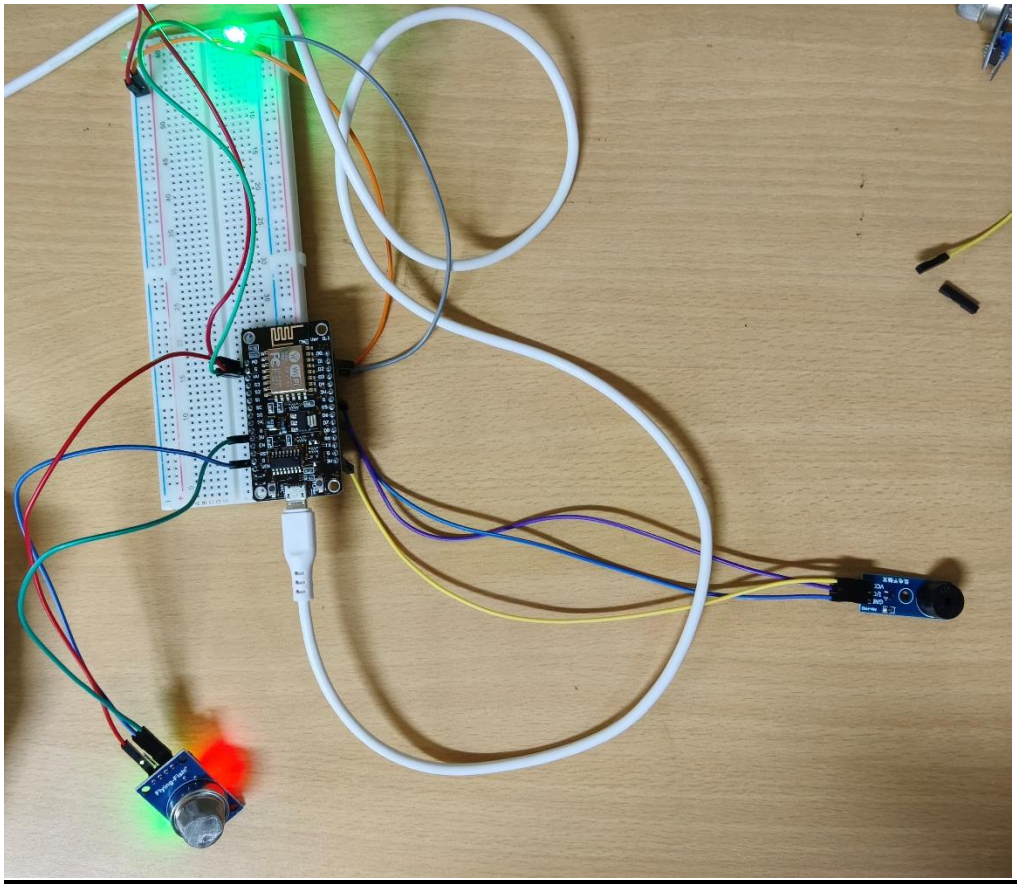
Step 11: Install the required OLED libraries.

Step 12: Upload the program.

Step 13: Allow the MQ-2 sensor to warm up.

Step 14: Observe the sensor reading on the OLED and Serial Monitor.

Step 15: Test the system under controlled conditions.



RESULTS & PERFORMANCE ANALYSIS:

The developed prototype successfully demonstrates the detection of changes in smoke/gas levels using the MQ-2 sensor.

The ESP8266 processes the sensor output and provides the corresponding indication through the OLED display, LEDs and buzzer.

The Air Pollution Monitoring System using MQ-2 Gas Sensor and ESP8266 NodeMCU was designed to detect changes in smoke and combustible gas levels in the surrounding environment.

The MQ-2 sensor continuously monitors the air and provides an analog signal to the ESP8266. The controller processes the signal and displays the sensor value and status on the OLED/LCD. The two LEDs provide visual indication, while the buzzer produces an alarm when the gas level exceeds the selected threshold.

The use of **ESP8266 NodeMCU** makes the project suitable for future IoT development because Wi-Fi connectivity can be used to send sensor data to a web server or cloud platform.